

Audion Image Tools

Портативный набор для конверсии и трансформации изображений на базе Audion Python Portable Template.

Главная практическая логика проекта:

- принимать и по возможности конвертировать максимум реальных входных форматов
- самую глубокую, безопасную и предсказуемую обработку сосредоточить вокруг JPG и PNG

`launcher_tools.cmd` оставлен без изменений из шаблона. Главная точка входа для ежедневной работы — `launcher_gui.cmd`. CMD-лаунчеры `launcher_project.cmd` и `launcher_project_ru.cmd` сохранены для привычных CLI/TUI-сценариев и работают в двойном режиме:

- главное меню использует FZF, если он доступен
- если FZF нет, выполняется fallback на обычное CMD-меню
- для критичных коротких выборов launcher использует обычный `Select` или ручной ввод, чтобы избежать путаницы значений на глубоких шагах

Что умеет проект

- конвертация поддерживаемых файлов в JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC
- основная конвертация: быстрые пресеты качества 60/75/90 плюс точный спиннер 1..100; значение по умолчанию 83
- для PNG в CLI есть служебный параметр сжатия, при этом GUI показывает простой пресет `PNG`
- вывод в WebP / AVIF / HEIF / HEIC для компактной доставки без замены JPG как совместимого fallback-формата
- исправление EXIF rotation
- запись DPI без изменения пикселей
- нормализация цвета с явным выбором профиля

- выбор для sRGB: Pillow `Create Profile sRGB` или `sRGB IEC 61966-2.1` с color.org
- выбор для CMYK: `Photoshop5DefaultCMYK.icc` или `CoatedFOGRA39.icc`
- безопасная обработка alpha для JPG
- перевод в оттенки серого
- подгонка под экран 1080p / 1440p / 2160p
- подгонка под 16:9, A4 и A3
- подгонка под рулон: пиксели не меняются, DPI рассчитывается по выбранной стороне в миллиметрах
- DPI: пиксели не меняются, записывается выбранный DPI
- обрезка однотонной рамки
- умный кроп белого фона
- защитное поле в мм после кропа, если нужен технологический запас
- сборка PDF сразу после кропа в режимах `Встроенный PNG` или `Встроенный JPG 75%`
- водяной знак: горизонтальная подпись в углу или диагональная защитная надпись примерно на 60% диагонали, с регулируемой прозрачностью и цветом
- замощение A5/A4/A3 или рулона одной картинкой для наклеек с полями, зазором и рамкой в мм
- генерация контактного листа с радиовыбором `Из списка` / `Свой размер`, dropdown типовых размеров миниатюры, выбором формата и качества 75/92
- разрезание многокадрового TIFF в нумерованные PNG-кадры
- разрезание изображения на ленты или сетку прямоугольников, до 100 частей, с PNG по умолчанию и выбором формата
- уменьшение до 25%, 50%, своего процента или автоматический подбор под целевой размер файла в MB
- выбор алгоритма уменьшения: `Lanczos`, `Bicubic`, `Box`, `Nearest`; `Box` и `Nearest` особенно полезны для PNG-карт, планов и плоской графики
- простой экспорт PDF в JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC с режимами встроенных растров (`embedded`) и полного рендеринга (`render`)
- `pdf-export --mode render` поддерживает произвольный DPI от `1` до `1200`
- растривание PDF-страниц в JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC при 150 / 300 / 600 DPI
- извлечение встроенных изображений из PDF как есть или сразу в JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC
- сборка PDF из картинок в режиме lossless-preferred или JPG 75% качества / 90% качества

- сборка PDF из картинок с опциональным произвольным DPI для физического размера страниц
- мост нормализации PDF: PDF -> TIFF-страницы -> нормализация -> PDF

Цветовые сценарии

Это самая важная часть, если нужно понять, какой режим выбирать.

Вывод в sRGB

Используется для экрана, web, обычной RGB-доставки и общей нормализации под экранный просмотр.

В проекте есть два явных варианта:

- `Create Profile sRGB (Pillow)`
- `sRGB IEC 61966-2.1 (color.org)`

Вывод в CMYK

Используется для печатно-ориентированного результата, когда нужен именно перевод в CMYK, а не просто обычный RGB-выход.

Сейчас в проекте доступны два явных CMYK-профиля:

- `Photoshop5DefaultCMYK.icc`
- `CoatedFOGRA39.icc`

Transform vs embed

В проекте теперь явно разведены две разные задачи, которые часто ошибочно смешивают:

1. использовать ICC-профиль как математику преобразования цвета
2. вшивать ICC-профиль в выходной файл как metadata blob

Это особенно важно для печати и PDF.

Текущая рабочая политика такая:

- при нормализации в `sRGB` проект пересчитывает пиксели и по умолчанию вшивает итоговый sRGB-профиль

- при нормализации в **CMYK** проект пересчитывает пиксели через выбранный CMYK ICC-профиль, но по умолчанию не вшивает этот CMYK-профиль в каждый выходной файл

Почему это сделано именно так:

- цветовое преобразование — это то, что реально меняет сами пиксели
- встроенный ICC-профиль — это уже описание для интерпретации этих пересчитанных пикселей
- большие CMYK ICC-профили могут сильно раздувать батчи, Office-файлы и растровые части PDF, если пришивать их к каждому выходному изображению

Для печати главный практический критерий не в том, "есть ли у каждого файла пришитый ICC-хвост", а в том, корректно ли напечатаны уже пересчитанные пиксели в целевом workflow.

Политика для PDF и печати

Для PDF действует та же логика разделения transform и embed.

- страницы PDF растрируются в TIFF
- TIFF-страницы нормализуются через выбранный профиль sRGB или CMYK
- затем из этих нормализованных страниц собирается новый PDF

По умолчанию проект старается сохранять профили у картинок, которые участвуют в PDF-сборке, потому что это может быть полезно в downstream-сценариях печати.

При этом:

- embed для **CMYK** остаётся опциональной, а не обязательной политикой
- базовая настройка предпочитает корректное преобразование в **CMYK** без принудительного вшивания тяжёлого ICC-блока в каждый выходной файл
- если конкретный печатный пайплайн действительно требует встроенный ICC-профиль внутри PDF-картинок, это можно включить отдельно от самой цветовой трансформации

Нормализация PDF

Нормализация PDF здесь не сводится к простой правке метаданных и не делает вид, будто PDF обрабатывается "как картинка" без промежуточных шагов.

Внутренний маршрут такой:

1. PDF растрируется в TIFF-страницы
2. TIFF-страницы нормализуются по выбранному пути sRGB или CMYK

3. из нормализованных страниц собирается новый PDF

Стратегия по форматам

Stable core

- BMP
- GIF
- JPG / JPEG
- PNG
- TGA
- TIFF / TIF
- WebP
- AVIF
- PSD как best-effort через Pillow

Extended adapters

- HEIC / HEIF через `pillow-heif`
- CR2 / CR3 / DNG через `rawpy`

Best-effort only

- WMF
- IFF
- XIF

Часть форматов требует optional-адаптеров или платформенных возможностей. Проверка статуса:

```
python system_core\main.py formats
```

Основные папки

- `input\` — быстрая папка источников
- `output\` — результаты обработки

- `logs\` — журналы запусков
- `report\` — подробные GUI-отчёты
- `workspace\` — временная рабочая область
- `config\` — defaults, GUI-темы и пути к ICC-профилям
- `system_core\` — Python-реализация
- `install\` — портативное окружение и вспомогательные скрипты
- `wheelhouse\` — offline wheels
- `release\` — архивы релизов

Лаунчеры

Запуск GUI:

```
launcher_gui.cmd
```

Русский TUI/CLI-лаунчер:

```
launcher_project_ru.cmd
```

Английский TUI/CLI-лаунчер:

```
launcher_project.cmd
```

Сначала лаунчер ищет встроенный `fzf.exe`, затем `fzf` из `PATH`, после чего при необходимости переходит на CMD fallback.

GUI-логика

В GUI команды сгруппированы по модулям:

- Конвертация
- PDF
- Обрезка
- Размер
- Цвет

- Фото-лист
- Листы и метки
- Контактный лист
- Диагностика

Верхний Workbench задаёт маршруты **ИСТОЧНИК** и **НАЗНАЧЕНИЕ**; внутренние папки проекта по-прежнему называются **input** и **output**. **ИСТОЧНИКОМ** может быть папка или один файл, выбранный кнопкой **ДОБАВИТЬ ФАЙЛ...** без копирования. **СБРОСИТЬ** сохраняет pin, убирает незакреплённый кэш и всегда возвращает активные маршруты к внутренним **input/output**; файлы при этом не удаляются. **УДАЛИТЬ** очищает содержимое обоих текущих маршрутов после общего подтверждения.

Конвертация открывается сразу как рабочее окно. Быстрые действия находятся сверху по сетке и запускаются сразу после нажатия. Обычная конвертация с выбором форматов, качества и DPI находится в этом же окне ниже. Команды с параметрами показывают логические прямоугольные блоки полей и янтарную кнопку **ЗАПУСТИТЬ**.

Тема GUI выбирается в шапке окна. Состояние хранится в **config/gui_settings.yaml**, а палитры и CSS-токены - в **config/ui_colors.yaml**.

Пользовательский порядок модулей: **Конвертация**, **PDF**, **Обрезка**, **Размер**, **Цвет**, **Фото-лист**, **Листы и метки**, **Контактный лист**, **Диагностика**.

Обслуживание вынесено в отдельный блок и требует подтверждения для опасных действий. Там же находится глобальный дефолт многопоточности: на машинах с **16 GB RAM** его лучше оставлять выключенным или ставить **1-2** потока; для **32 GB** обычно достаточно **4**, для **64 GB+** можно пробовать **8-12** после контроля пика памяти. Цветовая нормализация находится в отдельном модуле **Цвет** после **Размер**. Команды печати и плоттера находятся в модуле **Размер** как **Под рулон** и **DPI**. **Контактный лист** вынесен в ROOT над диагностикой как вспомогательная обзорная команда с подписями: имя файла, строка размера **px | cm | DPI** и KB; по умолчанию пишет PNG, но формат, качество 75/92 и размер миниатюры из dropdown или через явный режим **Свой размер** можно выбрать.

Короткие GUI-подписи раскрываются через tooltip-подсказки. Tooltip есть у заголовков блоков, кнопок, чекбоксов, radio и segmented toggle; **НАЗАД** подсказку не использует.

Текущая launcher-логика intentionally упрощена под ежедневную работу:

- launcher читает входные файлы из **input**
- launcher пишет результаты в стандартные подпапки внутри **output**

- для ручной работы с проводником есть отдельные пункты **Открыть INPUT** и **Открыть OUTPUT**
- PNG в GUI идет как простой быстрый пресет без раскрытия служебных параметров сжатия
- в экспорте страниц PDF доступны встроенные растры (**embedded**), быстрый рендеринг **render (300 DPI)** и **render** с ручным вводом своего DPI

Нормализация в лаунчере теперь намеренно упрощена:

- **Нормализация в sRGB, PDF остаётся PDF** спрашивает, какой вариант sRGB использовать
- **Нормализация в CMYK, PDF остаётся PDF** спрашивает, какой CMYK-профиль использовать
- старый широкий пункт с выбором параметров нормализации убран из меню как сбивающий с толку

То есть выбор профиля теперь делается прямо в момент выбора **sRGB** или **CMYK**, а не прячется в более абстрактном "расширенном" пункте.

Примеры CLI

Конвертировать папку в JPG с качеством по умолчанию 83%:

```
python system_core\main.py convert --input input --output output\jpg_83 --to jpg
```

Конвертировать папку в PNG:

```
python system_core\main.py convert --input input --output output\png --to png
```

Нормализовать в sRGB JPG с белой подложкой alpha:

```
python system_core\main.py normalize --input input --output output\normalized_jpg --to jpg --target-profile srgb --srgb-profile-mode colororg --srgb-profile config/icc/sRGB2014.icc --alpha-bg white --metadata preserve_dpi_color
```

Нормализовать в sRGB JPG через встроенный профиль Pillow:

```
python system_core\main.py normalize --input input --output output\normalized_jpg_pillow --to jpg --target-profile srgb --srgb-profile-mode pillow --alpha-bg white --metadata preserve_dpi_color
```


Нормализовать в CMYK TIFF через Photoshop 5 Default CMYK:

```
python system_core\main.py normalize --input input --output  
output\normalized_cmyk_ps5 --to tiff --target-profile cmyk --cmyk-profile  
config/icc/Photoshop5DefaultCMYK.icc --alpha-bg white --metadata preserve_dpi_color
```

Нормализовать в CMYK TIFF через Coated FOGRA39:

```
python system_core\main.py normalize --input input --output  
output\normalized_cmyk_fogra39 --to tiff --target-profile cmyk --cmyk-profile  
config/icc/CoatedFOGRA39.icc --alpha-bg white --metadata preserve_dpi_color
```

Записать DPI 300 без изменения пикселей:

```
python system_core\main.py normalize --input input --output output\dpi_300 --to png -  
-target-profile keep --metadata preserve_dpi_color --set-dpi 300
```

Подогнать к 2160p с обрезкой:

```
python system_core\main.py resize-screen --input input --output  
output\screen_2160_crop --to jpg --target 2160p --mode crop
```

Уменьшить PNG-карты до 50% без создания новых цветов:

```
python system_core\main.py downscale-percent --input input --output  
output\downscale_png_nearest_50 --to png --scale-mode percent-50 --algorithm nearest
```

Уменьшить карты до своего процента через сухое усреднение **Box**:

```
python system_core\main.py downscale-percent --input input --output  
output\downscale_png_box_custom --to png --scale-mode custom --custom-percent 37.5 --  
algorithm box
```

Подобрать JPG примерно до 1 MB на файл:

```
python system_core\main.py downscale-percent --input input --output  
output\downscale_jpg_1mb --to jpg --scale-mode target-mb --target-mb 1.0 --algorithm  
bicubic --jpeg-quality 75 --workers 8
```

Добавить диагональный защитный водяной знак примерно на 60% диагонали:

```
python system_core\main.py watermark --input input --output output\watermark_diagonal
--to png --text "DRAFT" --watermark-mode diagonal --diagonal-coverage 60 --opacity 96
--color "#cc0000"
```

Подгонка под рулон по короткой стороне 900 мм без изменения пикселей:

```
python system_core\main.py plotter-size --input input --output
output\plotter_roll_fit --to png --side short --target-mm 900
```

DPI без изменения пикселей:

```
python system_core\main.py plotter-dpi-only --input input --output
output\plotter_dpi_only --to png --dpi 300 --metadata preserve_dpi_color
```

Разрезать изображение на 4 вертикальные ленты:

```
python system_core\main.py split-half --input input --output output\split_png --mode
strips --orientation vertical --parts 4 --metadata preserve_dpi_color
```

Разрезать изображение сеткой 2 на 3:

```
python system_core\main.py split-half --input input --output output\split_png --mode
grid --rows 2 --columns 3 --metadata preserve_dpi_color
```

Разрезать TIFF в нумерованные PNG:

```
python system_core\main.py split-tiff --input input --output output\tiff_split
```

Замостить A4 наклейками 40x30 мм с полями 7 мм и рамкой 0.2 мм:

```
python system_core\main.py tile-sheet --input input --output output\tile_sheet --
paper a4 --orientation portrait --margin-mm 7 --gap-mm 0 --dpi 300 --size-mode doc-
40x30 --frame-mm 0.2 --count-mode fill
```

Сделать рулон шириной 600 мм на точные 3 ряда:

```
python system_core\main.py tile-sheet --input input --output output\tile_roll --paper roll --roll-width-mm 600 --margin-mm 7 --gap-mm 0 --dpi 300 --size-mode width --item-width-mm 40 --count-mode rows --rows 3
```

Обрезать белый фон точно по значимым пикселям:

```
python system_core\main.py smart-crop-white --input input --output output\smart_crop_white --to png --tolerance 10 --safety-margin-mm 0
```

Обрезать однотонную рамку с защитным полем 2 мм:

```
python system_core\main.py trim-border --input input --output output\trimmed --to png --tolerance 10 --safety-margin-mm 2
```

Растривать страницы PDF в PNG при 300 DPI:

```
python system_core\main.py pdf-rasterize --input input\sample.pdf --output output\pdf_pages --to png --dpi 300
```

Экспортировать PDF в PNG через самый крупный встроенный растр на каждой странице:

```
python system_core\main.py pdf-export --input input\sample.pdf --output output\pdf_export --to png --mode embedded
```

Экспортировать PDF в PNG как полный рендер страницы с текстом и вектором:

```
python system_core\main.py pdf-export --input input\sample.pdf --output output\pdf_export_render --to png --mode render --dpi 300
```

Экспортировать PDF в PNG как полный рендер страницы с произвольным DPI, например 200:

```
python system_core\main.py pdf-export --input input\sample.pdf --output output\pdf_export_render_200 --to png --mode render --dpi 200
```

Извлечь встроенные изображения из PDF без рендера страницы:

```
python system_core\main.py pdf-extract-embedded --input input\sample.pdf --output output\pdf_embedded_extract
```

Извлечь встроенные изображения сразу в JPG в одну папку:

```
python system_core\main.py pdf-extract-embedded --input input --output output\pdf_embedded_jpg --to jpg --layout flat
```

Извлечь встроенные изображения в PNG и, если внутри CMYK/ICC, положить рядом TIFF:

```
python system_core\main.py pdf-extract-embedded --input input --output output\pdf_embedded_png --to png --layout flat --png-side-tiff true
```

Извлечь встроенные изображения в TIFF, что удобно для CMYK-вложений:

```
python system_core\main.py pdf-extract-embedded --input input --output output\pdf_embedded_tiff --to tiff --layout flat
```

Собрать PDF из картинок в режиме lossless-preferred:

```
python system_core\main.py images-to-pdf --input input --output output\images_to_pdf.pdf --mode lossless
```

Собрать PDF из картинок с физическим размером страниц по 300 DPI:

```
python system_core\main.py images-to-pdf --input input --output output\images_to_pdf.pdf --mode lossless --dpi 300
```

Примечания

- JPG не является математически lossless-форматом. Когда нужна работа без потерь, лучше использовать PNG.
- `plotter-size` и `plotter-dpi-only` являются неразрушающими по пикселям: первая команда рассчитывает DPI по миллиметрам, вторая записывает выбранный DPI.
- `trim-border` и `smart-crop-white` при `--safety-margin-mm 0` режут ровно по внешнему периметру значимых пикселей; значение больше нуля оставляет исходный фон вокруг

результата по DPI изображения.

- После кропа GUI может сразу собрать PDF без изменения пикселей: **Встроенный PNG** для lossless-ориентированной сборки или **Встроенный JPG 75%** для компактного PDF.
- **split-half** сохранил старое имя CLI ради совместимости, но теперь умеет ленты и сетку. Файлы частей пишутся плоско в output с суффиксом **_01** ... **_100**; PNG остается дефолтом, формат можно выбрать.
- **pdf-export --mode embedded** берёт самый крупный встроенный растр на странице и особенно удобен для одностраничных PDF-сканов, внутри которых фактически лежит один большой JPG.
- **pdf-export --mode render** рендерит всю страницу целиком и сохраняет видимый текст и вектор в выбранном растровом формате.
- Для **pdf-export --mode render** можно задавать произвольный положительный DPI до **1200**.
- Растрирование PDF всегда рендерит внешний вид всей страницы; извлечение встроенных изображений достаёт только внутренние растровые объекты PDF.
- **pdf-extract-embedded --layout flat** пишет извлеченные картинки в одну папку; **--layout folders** пишет в подпапки по именам PDF.
- **pdf-extract-embedded --to png --png-side-tiff true** пишет дополнительный TIFF рядом с PNG, если встроенный растр CMYK или содержит ICC.
- Отчет **pdf-extract-embedded** добавляет **output_details** с форматом, режимом изображения и именем ICC-профиля, если профиль удалось прочитать.
- **images-to-pdf --dpi** задает PDF page size через DPI без изменения пикселей исходных картинок.
- **photo-sheet-width** по умолчанию не пересчитывает пиксели исходных картинок: ширина листа и DPI задают итоговый лист. Если при выбранной ширине DPI слишком мал для исходных пикселей, движок автоматически повышает DPI листа и пишет предупреждение в отчёт.
- Нормализация PDF работает постранично: PDF растрируется в TIFF, страницы нормализуются, затем собирается новый PDF.
- ICC-преобразование и ICC-вшивание — это разные политики: transform меняет пиксели, embed добавляет только профильные метаданные.
- Дефолтная политика: **sRGB** вшивается при сохранении, **CMYK** по умолчанию не вшивается, а в PDF-сценариях профили у картинок сохраняются там, где это полезно.

- Для многостраничного PDF и TIFF экспорт страниц идёт с префиксом `001_name` ... `999_name`.
- Для одностраничного PDF и TIFF префикс номера страницы не добавляется.
- Набор ICC-профилей в проекте сейчас намеренно минимальный: `sRGB2014.icc`, `Photoshop5DefaultCMYK.icc`, `CoatedFOGRA39.icc`.
- При обработке папки структура исходных подпапок сохраняется внутри выбранной output-папки.
- Launcher не спрашивает путь к input/output на каждом шаге: он использует проектные `input\` и `output\` по умолчанию и показывает их перед запуском команды.

Быстрые ориентиры

- Выбирай `sRGB`, когда результат нужен для экрана, сайта или обычной RGB-передачи.
- Выбирай `CMYK`, когда результат нужен для печатного сценария.
- Выбирай Pillow sRGB, когда нужен встроенный путь CMS без внешнего ICC-файла.
- Выбирай профиль color.org, когда нужен явный внешний sRGB-эталон.
- Выбирай `Photoshop5DefaultCMYK.icc` для широкого классического CMYK-сценария.
- Выбирай `CoatedFOGRA39.icc`, когда нужен именно этот профиль для покрытой печати, а не более широкий дефолтный CMYK-путь.